



IA & APRENDIZAJE AUTOMÁTICO EN CIENCIAS

Fundamentos Matemáticos, desarrollo de modelos
y aplicaciones en educación e investigación.

PROYECTO FINAL

Chatbot con Procesamiento de Lenguaje Natural (NLP)

Erick Samuel Martínez Calderón

Plantel Naucalpan

Agosto 2026

Introducción

El presente documento describe el desarrollo de un chatbot basado en Procesamiento de Lenguaje Natural (NLP), desarrollado como Proyecto Integrador final para el Diplomado de IA en CCH Azcapotzalco. El sistema está diseñado para resolver dudas administrativas de la comunidad docente (horarios, convocatorias, calendarios y trámites) a través de un chat accesible (WhatsApp), utilizando información extraída directamente de los documentos oficiales del plantel.

El proyecto combina técnicas de Recuperación Aumentada por Generación RAG con un modelo de lenguaje de gran escala (LLM) para producir respuestas basadas exclusivamente en el contenido institucional real, minimizando el riesgo de respuestas inventadas o inexactas (alucinaciones).

Objetivo

Diseñar e implementar un chatbot capaz de responder preguntas frecuentes de la comunidad estudiantil y administrativa sobre procesos, fechas y trámites del plantel, utilizando exclusivamente la información contenida en documentos oficiales (PDFs y calendario en formato CSV), y ponerlo a disposición de los usuarios a través de un canal de uso cotidiano como WhatsApp.

Pasos

El sistema sigue una arquitectura de tipo RAG (Retrieval-Augmented Generation), donde la generación de respuestas del modelo de lenguaje se apoya en información recuperada de una base de datos vectorial, en lugar de depender únicamente del conocimiento interno del modelo.

Flujo de datos

- Ingesta: los documentos PDF y el calendario CSV se procesan y se dividen en fragmentos de texto (chunks).
- Vectorización: cada fragmento se convierte en un vector numérico (embedding) que representa su significado semántico.
- Almacenamiento: los vectores se guardan en una base de datos vectorial local (ChromaDB).
- Consulta del usuario: la pregunta que llega por WhatsApp también se convierte en un vector.
- Recuperación: se buscan los fragmentos más similares semánticamente a la pregunta.
- Generación: esos fragmentos se envían como contexto al modelo de lenguaje (Groq / Llama 3.3), que redacta la respuesta final.
- Entrega: la respuesta regresa al usuario por WhatsApp a través de Twilio.

Pipeline

WhatsApp (usuario) → Twilio → Webhook Flask (ngrok) → Búsqueda semántica en ChromaDB → Generación de respuesta con Groq → Respuesta → Twilio → WhatsApp (usuario).

Herramientas utilizadas

A continuación se detalla cada componente tecnológico del proyecto, las alternativas que se consideraron, y el razonamiento detrás de la decisión final. La elección general priorizó tres criterios: costo (herramientas gratuitas o con capa gratuita suficiente para un proyecto académico), simplicidad de implementación, y que cada herramienta fuera adecuada al volumen y tipo de datos manejados (documentos institucionales en español, volumen moderado).

Componente	Alternativas consideradas	Por qué se eligió
PyPDF2	pdfplumber, PyMuPDF (fitz)	Es ligera, no requiere dependencias externas del sistema y es suficiente para extraer texto plano de PDFs institucionales que no tienen tablas complejas ni escaneos. Para el volumen de documentos del proyecto (reglamentos, convocatorias, horarios) su precisión es adecuada.
RecursiveCharacterTextSplitter (LangChain)	Fragmentación manual por longitud fija, NLTK sentence tokenizer	A diferencia de un corte fijo por número de caracteres, este divisor respeta la jerarquía natural del texto (párrafos, luego líneas, luego palabras), evitando cortar oraciones a la mitad. El traslape (chunk_overlap) conserva contexto entre fragmentos, mejorando la calidad de las respuestas del RAG.
ChromaDB	FAISS, Pinecone, Weaviate	Es una base de datos vectorial local y gratuita (PersistentClient), sin necesidad de servicios en la nube de pago ni configuración de infraestructura. Ideal para un proyecto académico donde no se requiere escalar a millones de documentos.
paraphrase-multilingual-MiniLM-L12-v2 (Sentence Transformers)	Modelos de embeddings solo en inglés, OpenAI Embeddings (de pago)	Es un modelo multilingüe preentrenado que funciona bien en español sin costo, corre localmente sin depender de una API externa, y su tamaño reducido (MiniLM) permite generar embeddings rápido incluso sin GPU dedicada.
Groq (Llama 3.3 70B Versatile)	Gemini, OpenAI GPT, Ollama local	Ofrece una capa gratuita con límites generosos y tiempos de respuesta muy rápidos gracias a su infraestructura especializada (LPU). A diferencia de un modelo local (Ollama), no requiere que la computadora tenga recursos de cómputo dedicados para generar las respuestas.
Flask	FastAPI, Django	Es un microframework minimalista, suficiente para exponer un único endpoint (webhook) sin la complejidad de un framework completo como Django. Su curva de aprendizaje es corta y es el estándar más común para prototipos de este tipo.
Twilio (WhatsApp Sandbox)	WhatsApp Cloud API (Meta) directo, librerías no oficiales (whatsapp-web.js)	Permite conectar un chatbot a WhatsApp en minutos sin pasar por el proceso de verificación de negocio de Meta, y sin violar los términos de servicio de WhatsApp (como sí ocurre con librerías no oficiales que simulan WhatsApp Web).
ngrok	Hosting en la nube (Render, Railway)	Permite exponer el servidor Flask local a internet con una URL pública HTTPS de forma inmediata, sin necesidad de desplegar el proyecto en un servicio externo mientras se está en fase de pruebas y desarrollo.

Retos

Durante el desarrollo del proyecto se presentaron los siguientes problemas técnicos, documentados junto con su diagnóstico y solución:

Reto encontrado	Solución aplicada
Exposición accidental de la API key de Groq	Se movió la clave a un archivo .env cargado con python-dotenv, evitando que quede escrita directamente en el código o en el notebook.
Respuesta 403 Forbidden desde ngrok hacia el webhook de Twilio	Se identificó que el puerto 5000 estaba ocupado por el servicio AirPlay Receiver de macOS. Se cambió Flask al puerto 5001 y se reconfiguró ngrok, resolviendo el conflicto.
Advertencia de seguridad del plan gratuito de ngrok bloqueando peticiones automatizadas	Se agregó el parámetro ?ngrok-skip-browser-warning=true a la URL configurada en Twilio para omitir la pantalla de advertencia en peticiones no interactivas.

Consideraciones

- Las credenciales de API (Groq) se manejan mediante variables de entorno (.env), nunca escritas directamente en el código fuente.
- El archivo .env se excluye de cualquier control de versiones o entrega pública del proyecto.
- El sistema RAG restringe las respuestas del modelo únicamente al contexto recuperado de los documentos oficiales, reduciendo el riesgo de que el chatbot proporcione información incorrecta o fuera de su alcance.

Conclusiones

El proyecto demuestra la integración práctica de varias técnicas de Procesamiento de Lenguaje Natural extracción de texto, fragmentación, embeddings semánticos, recuperación vectorial y generación aumentada en un producto funcional y accesible para la comunidad del plantel. La arquitectura RAG permite mantener actualizada la base de conocimiento simplemente agregando o reemplazando documentos, sin necesidad de reentrenar ningún modelo. El uso de herramientas gratuitas y de bajo costo de configuración (ChromaDB, Groq, Twilio Sandbox, ngrok) hizo viable la implementación completa dentro de las limitaciones de tiempo y recursos de un proyecto académico.

Mejoras

- Migrar de Twilio Sandbox a la API oficial de WhatsApp Business (Meta Cloud API) o a un número de Twilio de producción para uso más allá de pruebas.
- Desplegar el servidor Flask en un hosting permanente en la nube (por ejemplo, Render o Railway) para evitar depender de ngrok y de una URL que cambia en cada sesión.
- Incorporar un clasificador de intención basado en Machine Learning (TF-IDF + Regresión Logística) para enrutar las preguntas por categoría antes de la búsqueda vectorial.
- Agregar un mecanismo de retroalimentación de los usuarios para medir la satisfacción con las respuestas generadas.